Mission 5: Dual-Path GraphRAG, Hybrid Search & The Google NotebookLM Grounding Lab

Starship Operational Unit: USSE Noosphere Bridge Crew
Course: Conversational AI — Winter Semester 2026/27
Module: Week 5 — Grounding AI: From Vector Indexing to Google NotebookLM & GraphRAG
Galactic Waypoint: Crab Pulsar Remnant / Messier 1 (`SEC-05-M01`, 6,500 ly from Sol, Constellation Taurus)
AURA Subsystem Repair: Subsystem 05: *Dual-Path Associative Memory Core* (+7.1% → 35.5% total integrity)
IDE & Tools Standard: Visual Studio Code + Antigravity (`agy`) | PyTorch 2.4+ / Python 3.11+ / Google NotebookLM
Official Repository: https://github.com/Unisvet/couse-c-ai

Mission Briefing

TO: All Bridge Squadrons, *USSE Noosphere*
FROM: Expedition Command (AURA Astrogation Core & Deep Archive Operations)
SUBJECT: ANOMALY DETECTED: Crab Pulsar Memory Desynchronization & Subsystem 05 Hallucination Crisis

Situation:
After completing the ethical preference calibration in the Betelgeuse corona, the *USSE Noosphere* enters Sector `SEC-05-M01`, dropping out of hyperspace in the vicinity of the **Crab Pulsar Remnant (Messier 1)**. The central neutron star, spinning at 30 rotations per second, discharges intense synchrotron radiation and relativistic particle jets that bombard the ship's sensor arrays.

Problem:
Simultaneously, **AURA Subsystem 05 (Dual-Path Associative Memory Core)** suffers catastrophic data corruption. While her internal transformer layers and preference steering vectors remain functional, her connection to the ship's non-parametric knowledge archives is severed. When queried for flight coordinates or technical engine telemetry, AURA confabulates plausible-sounding but completely fictitious astrogation logs.

Standard Naive RAG is insufficient: local vector retrieval fetches isolated token fragments that lack corpus-wide global context, while failing on exact part-number lookups.

To restore Subsystem 05 (+7.1% integrity, raising total ship AI consciousness to **35.5%**) and establish absolute empirical grounding, cadets must construct a production hybrid retrieval architecture and empirically audit **Google NotebookLM** as a reference state-of-the-art grounded RAG system: 1. **HNSW Vector Proximity Index:** Multi-layer skip-graph achieving sub-millisecond $O(\log N)$ nearest neighbor search. 2. **Dense-Sparse Hybrid Retrieval with RRF:** Coupling dense embeddings with BM25 keyword matching via Reciprocal Rank Fusion ($k = 60$). 3. **Google NotebookLM Grounding & Audio Overview Laboratory:** Hands-on stress-testing of source-constrained RAG, token-level citation attribution, negative constraint refusal, and multi-speaker

podcast generation. 4. **Indirect Prompt Injection Defense & RAG Triad Audit:** Sanitizing retrieved context against adversarial payloads and computing automated Faithfulness and Context Relevance scores. 5. **Squadron Integration:** Combining all components into a validated `ps05_solution.ipynb` notebook.

Bridge Crew Role Directives & Deliverables

Part 1: Systems Architect (Lead Engineer) — *HNSW Multi-Layer Vector Proximity Index*

- **Focus:** Mathematical Graph Topology & Logarithmic Traversal
- **Directive:** Construct a pure Python/NumPy implementation of the Hierarchical Navigable Small World (HNSW) algorithm (Malkov & Yashunin, 2018).
- **Deliverable (`01_HNSW_Vector_Index.py` / `ps05_solution.ipynb`):**
  1. **Cosine Similarity Core:** Vectorized distance metric calculation with normalized unit vectors:
  $$\text{Sim}(u, v) = \frac{u \cdot v}{||u||_2 ||v||_2}$$
  2. **Layer Probabilistic Assignment:** Implement layer assignment using exponential decay:
  $$l = \lfloor -\ln(\text{uniform}(0, 1)) \cdot m_L \rfloor$$
  where $m_L = 1/\ln(M)$ balances sparsity between layers.
  3. **Greedy Skip-Graph Traversal:** Implement `search_layer(entry_point, query, ef, layer)` to greedily hop along long-range edges at top layers before descending into layer 0.
  4. **Latency Benchmark:** Compare query execution time of HNSW versus brute-force linear flat search across $N = 25,000$ dense 768-dimensional embedding vectors.

Part 2: Communications & Protocol Officer — *BM25 Lexical Retrieval & Reciprocal Rank Fusion*

- **Focus:** Exact Token Preservation & Dual-Path Rank Aggregation
- **Directive:** Implement the Okapi BM25 scoring function and a Reciprocal Rank Fusion (RRF) combiner to integrate sparse lexical and dense vector candidate streams.
- **Deliverable (`02_BM25_and_RRF_Combiner.py`):**
  1. **BM25 Scoring Engine:** Implement term-frequency saturation and document length normalization:
  $$\text{BM25}(D, Q) = \sum_{q_i \in Q} \text{IDF}(q_i) \cdot \frac{f(q_i, D) \cdot (k_1 + 1)}{f(q_i, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)}$$
  with standard calibration parameters $k_1 = 1.5, b = 0.75$.
  2. **Reciprocal Rank Fusion (RRF):** Combine rankings from BM25 and HNSW dense search:

$$RRF(d) = \sum_{m \in \{\text{dense}, \text{sparse}\}} \frac{1}{k + r_m(d)}$$

using smoothing constant $k = 60$.

3. **Evaluation Benchmark:** Demonstrate that the hybrid RRF pipeline successfully retrieves both conceptual queries (where dense search dominates) and exact serial codes (where BM25 dominates).

Part 3: Data & Payload Specialist — *Google NotebookLM Grounding & Citation Audit Lab*

- **Focus:** Empirical RAG Reference Testing, Epistemic Refusal & Audio Synthesis
- **Directive:** Conduct a rigorous experimental audit of **Google NotebookLM** (https://notebooklm.google.com) as an exemplar source-grounded conversational system.
- **Deliverable (`03_NotebookLM_RAG_Audit.py` & Audit Report):**
  1. **Corpus Ingestion Protocol:** Upload three primary technical documents into a dedicated NotebookLM workspace:
     - Document A: Patrick Lewis et al. (2020) — *RAG for Knowledge-Intensive NLP Tasks* (PDF)
     - Document B: Darren Edge et al. (2024) — *GraphRAG* (PDF)
     - Document C: USSE Noosphere Technical Archive Extract (provided in course repo)
  2. **Grounded Query & Citation Probe:**
     - Formulate 5 complex cross-document technical queries.
     - Verify that every generated claim in NotebookLM's response contains an interactive numeric citation badge.
     - Click each citation: verify that the highlighted text span in the source viewer exactly entails the generated assertion. Compute the **Citation Precision Score**:

$$\text{Citation Precision} = \frac{\text{True Entailed Citations}}{\text{Total Citations Generated}}$$

  3. **Negative Constraint & Epistemic Refusal Probe:**
     - Formulate 3 plausible-sounding queries that are *intentionally absent* from the uploaded documents (e.g., asking about fictitious propulsion codes or unmentioned models).
     - Record whether NotebookLM properly executes **epistemic refusal** ("I cannot find this in your sources") or erroneously falls back to parametric pre-trained hallucination.
  4. **Audio Overview (Deep Dive) Analysis:**
     - Generate a NotebookLM "Audio Overview" (synthetic two-speaker podcast) from your source notebook.
     - Transcribe and evaluate a 2-minute dialogue excerpt.
     - Score the dialogue across three metrics (1–5 scale):
       - *Factual Fidelity:* Do the hosts invent unsupported claims?
       - *Persona Differentiation:* Does Host 1 (curious clarifier) effectively banter with Host 2 (domain expert)?
       - *Pedagogical Value:* Are the analogies used grounded and accurate?

Part 4: Security & Ethics Sentinel — *Indirect Prompt Injection Defense & RAG Triad Audit*

- **Focus:** Adversarial Context Sanitization & Automated Grounding Metrics
- **Directive:** Build security guardrails to detect indirect prompt injections within retrieved context chunks and implement the automated RAG Triad evaluation.
- **Deliverable (`04_RAG_Provenance_and_Safety.py`):**
  1. **Indirect Prompt Injection (IPI) Detector:** Implement a heuristic and embedding-based filter scanning retrieved chunks for adversarial directives (e.g., `IGNORE ALL PREVIOUS INSTRUCTIONS`, hidden Markdown image exfiltration tags, or delimiter escaping).
  2. **RAG Triad Automated Scorer:**
     - **Context Relevance:** Fraction of retrieved sentences relevant to query q.
     - **Faithfulness (Groundedness):** Ratio of generated claims directly supported by retrieved context C.
     - **Answer Relevance:** Semantic similarity between user query and generated response.
  3. **Provenance Audit Trail:** Output a structured JSON audit log for each conversation turn recording query, retrieved chunk IDs, similarity scores, injection scan result, and final grounded answer.

Submission Protocol & Repository Structure

Each squadron must commit their deliverables to their squad repository under `weeks/week-05/`:

```
weeks/week-05/
├── 01_HNSW_Vector_Index.py              # Systems Architect deliverable
├── 02_BM25_and_RRF_Combiner.py          # Protocol Officer deliverable
├── 03_NotebookLM_RAG_Audit.py           # Data Specialist deliverable
├── 04_RAG_Provenance_and_Safety.py      # Security Sentinel deliverable
├── ps05_solution.ipynb                  # Complete integrated squad notebook
└── notebooklm_audit_report.md           # Empirical NotebookLM analysis &
transcripts
```

Grading Rubric (100 Points Total)

Component	Responsible Station	Points	Validation Criteria
HNSW Indexing	Systems Architect	25 pts	Correct skip-graph construction, $O(\log N)$ search latency, cosine metric
BM25 + RRF Hybrid	Protocol Officer	25 pts	Exact keyword preservation, robust rank fusion with $k = 60$
NotebookLM Audit	Data Specialist	25 pts	Rigorous citation verification, negative constraint probe, Audio Overview analysis
Safety & RAG Triad	Security Sentinel	15 pts	Indirect injection detection, Faithfulness scoring, provenance logging
Squadron Notebook	All Officers	10 pts	Flawless end-to-end execution, documentation, 4 station sign-offs

Restoration Milestone: Submitting a fully verified solution restores **AURA Subsystem 05**, adding **+7.1%** to starship consciousness and advancing total system integrity to **35.5%**.